Patch-Aware KV Cache Compression for Byte-Level Language Models: A Controlled Study of Attention-Guided, Entropy-Guided and Recency Eviction

Aditya Rayaprolu, Kshitij Gupta, Dhwaj Kishore, Arnab Hazra, Riyazuddin Khan
CS F429 Natural Language Processing, BITS Pilani Hyderabad Campus, Group 12

Abstract

Byte-level language models avoid tokenizers but produce sequences about four times longer than BPE, so the key-value (KV) cache, the main deployment bottleneck, grows about four times faster. We build a 13.3M-parameter Byte Latent Transformer (BLT)-style model with patch-granular KV caches and evaluate cache compression under a controlled protocol: three seeds per setting, paired bootstrap intervals, random and recency controls, a held-out test split and an analytic compute count. Four results are positive. (i) Eviction plus 4-bit quantization keeps 7.4% of a WikiText cache for a cost of +0.0012 bits per byte (BPB). (ii) An online SnapKV-style policy, scored on the model's own attention, beats recency in 24 of 24 comparisons. (iii) A fixed-size cache follows documents four times longer than the training context, and 20 PG-19 books, better than resetting the context, with measured constant memory (0.59 MB against 136.5 MB) and constant time per byte. (iv) The BLT needs 2.8 times fewer FLOPs per byte than a byte-level Transformer of the same width. We also test a natural proposal, keeping the patches with high next-byte entropy, the signal the BLT already computes. It is worse than random in 68 and worse than recency in 70 of 72 comparisons, and the penalty grows about tenfold with ten times more training; an error analysis shows that it keeps mostly one- and two-byte fragments. We state what is new, what did not work, and what this scale cannot show.

1 Introduction

Task. We study language modelling: predicting the next symbol of a text given the previous ones, scored by bits per byte (BPB) on held-out documents. Production systems use subword tokenizers such as BPE [21]. Byte-level models remove the tokenizer and its failure modes on rare words, typos, code and other scripts [18, 22, 27, 28], but a byte sequence is about four times longer than its BPE sequence (4.0 bytes per token in our data).

Why the cache matters. When a decoder generates text it stores the keys and values of every earlier position in every layer. This KV cache, not the weights, limits batch size and context length at deployment. A byte-level Transformer of our width needs 151 MB per sequence at 16,384 bytes against 38 MB for BPE (Figure 8). The BLT [18] runs its large global Transformer over *patches* of bytes, so it keeps one cache entry per patch, and it already computes a next-byte entropy to decide where patches end.

Why we chose this problem. Our Part 1 proposal started from three observations: the KV cache, not the weights, limits how many users share a GPU and how long a context can be; tokenizer-free models are attractive for noisy text, code and morphologically rich languages but make that cache four times larger; and the entropy that the BLT uses to place patches is computed and then discarded, while existing eviction methods rely on recency or on attention scores computed at runtime. The problem is real, small enough to study from scratch on one GPU, and the answer is informative whichever way it comes out.

Question. Is that entropy also a good signal for deciding which patches to keep, or to store at low precision? It costs nothing extra, so a positive answer would be a free compression rule. Related work uses uncertainty to set cache budgets or as one term of a mixed score on subword models (Section 2); we found no test of entropy-ranked retention on byte-level models. We tested it against random, recency and attention-based rules at equal cache bytes. The answer is negative, and the same protocol produced four positive results (Table 1).

Contributions.

1. A BLT-style 13.3M-parameter byte model, compared with a Kneser-Ney byte 4-gram and a 13.8M BPE Transformer, three seeds per setting.
2. A controlled test of entropy-ranked cache retention on a byte-level patch model (a combination we found no prior test of), with a clear negative result, an explanation of what the rule actually keeps, and the checks that rule out the obvious escape routes.
3. Practical compression that works: attention-guided eviction, 4-bit quantization and bounded-memory streaming, with measured memory and time.
4. An online, tested implementation of H2O and SnapKV-style policies over patch caches, an analytic compute count, quantitative and qualitative error analysis, and an account of the problems we met and the limits that remain.

2 Literature survey

Tokens and bytes. BPE merges frequent byte pairs into a fixed vocabulary [21]. Token-free models work on bytes directly: ByT5 [27] shows that a standard Transformer on bytes is competitive in parameters, training compute and inference speed and more robust to noise; MegaByte [28] uses a local model inside patches and a global model between them; SpaceByte [22] applies larger blocks after word-boundary bytes such as spaces. The Byte Latent Transformer [18] forms patches dynamically from the next-byte entropy of a small byte model, runs a large latent Transformer over them, and reports a FLOP-controlled scaling study up to 8B parameters in which patches scale better than tokens at a fixed inference cost. Our model is a simplified BLT: mean pooling instead of cross-attention, no n-gram hash embeddings, and a 13.3M size.

KV cache eviction. StreamingLLM keeps a few initial "sink" positions and a recent window so a model can stream far past its training length [26]. H2O keeps "heavy

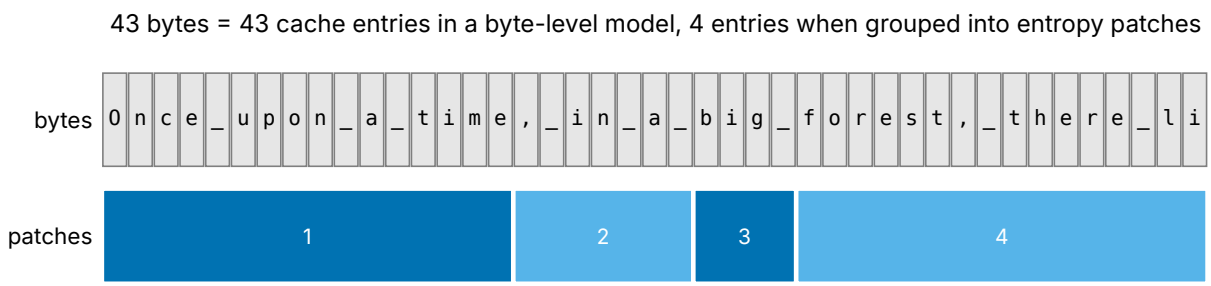


Figure 1. Why patches help the cache. The first 43 bytes of a real TinyStories document (spaces shown as underscores) and the four entropy patches the model forms over them. A byte-level model stores one key-value entry per byte; the BLT-style model stores one per patch.

Table 1. This work against the previous techniques. "Previous" is the Part 2 pipeline (n-gram and BPE Transformer), the standard recency (StreamingLLM-style) cache, context resets for long documents, and a byte-level Transformer. Evidence in the sections named in the last column.

Aspect	Previous technique	This work	Section
KV cache at 4,096 bytes	Byte-level Transformer 37.7 MB; BPE Transformer 9.4 MB (hypothetical length); BLT full cache 8.4 MB	0.2 MB with 64 patches in 4-bit (+0.0026 BPB on WikiText); 7.4% of the BLT cache with 128 patches in 4-bit for +0.0012 BPB	5.5
Eviction rule	Recency with a sink	Online SnapKV-style: better in 24 of 24 comparisons, by 0.0002 to 0.0077 BPB	5.5
Long documents	Reset the context every 4,096 bytes	Fixed recency cache: −0.0030, −0.0032, −0.0008 BPB (WikiText, 64 patches, three seeds); −0.0086 (20 PG-19 books, 256 patches); measured memory constant, time per byte 0.9 ms against a rise to 2.0 ms	5.5
Compute per byte	Byte-level Transformer, 30.9 MFLOPs (1,024 bytes)	BLT 11.1 MFLOPs, 2.8 times fewer (4.75 times at 4,096 bytes); analytic	5.5
Patching	Fixed stride 4 or 5	Entropy patching is lower in BPB in 3 of 3 same-seed pairs on WikiText validation and test, 67% of patch starts at word starts; gap about one seed range, so likely rather than established	5.2
Modelling quality	Kneser-Ney 4-gram 2.4103; BPE Transformer 1.5200 (WikiText val)	BLT 1.6801 at equal training (behind BPE); 1.29 to 1.31 with ten times more training	5.2
Entropy-guided retention (the proposal)	Random and recency	Not an improvement: worse than random in 68 and recency in 70 of 72 comparisons; penalty grows with training	5.3, 5.6

hitters", the positions with the largest accumulated attention [29]. SnapKV selects positions that the last queries of the prompt attend to [14]. All three act on subword models. We implement recency with a sink, H2O and a SnapKV-style rule (attention from the last 16 patches) as policies over patch entries.

KV quantization. KIVI quantizes keys per channel and values per token to 2 bits without tuning [15]; KVQuant adds non-uniform and outlier-aware schemes for very long contexts [9]. We use a simpler symmetric per-vector absmax quantizer and ask only whether the entropy signal can allocate bits.

Uncertainty in cache management. CONF-KV sets the per-step cache budget from the next-token entropy and ranks tokens by attention and recency, with random eviction under the same schedule as a control [13]. InfoKV mixes predictive entropy and layer-wise representation change with attention scores [10]. Counter-causal surprise scores cached tokens by how well later tokens predict them [5]. A learned controller chooses 2 to 16 bits per token using features that include entropy [7]. Scratchpad Patching uses next-byte uncertainty to add temporary slots inside byte patches and reports a smaller KV cache [30]. We read the first two in full and only the abstracts of the other three, so our statement of difference from those is provisional. In all of these entropy is one ingredient or sets a budget; none, to our knowledge, ranks which patches of a byte-level model to keep by entropy alone against random and recency at equal bytes.

Relation to widely used LLMs. Public general-purpose models read BPE subword tokens: Llama 2 uses a 32k vocabulary and 4k context, Llama 3 a 128k vocabulary and up to 128k tokens of context [6, 24]. They already fight cache growth in the architecture: Llama 3 uses grouped-query attention with 8 key-value heads "to reduce the size of key-value caches during decoding", which cuts the cache per token fourfold against one KV head per attention head for the 8B model (Figure 2). Eviction and quantization of the cache (Section 2, above) are research methods layered on top. We study the same bottleneck at a scale one GPU can train from scratch (13.3M parameters, about 600 times smaller than an 8B model) and in the setting these models avoid, raw bytes. Our models do not match a chat model in capability and we do not compare quality; what transfers is the measurement protocol, the policies, and the finding that attention-guided rules help while the free entropy signal does not. Because a byte model needs about four times more positions for the same text, its cache problem is worse, and patching (BLT) is the architectural answer that our compression methods then build on.

Classical baseline. Interpolated Kneser-Ney smoothing [2, 11] is the strongest classical n-gram method and gives a floor that any neural model should beat.

3 Dataset

Sources and fit. TinyStories [4] is a synthetic corpus of short stories generated by GPT-3.5 and GPT-4; it trains small models quickly and gives short documents (about 890 bytes). WikiText-103 [17] holds long Wikipedia arti-

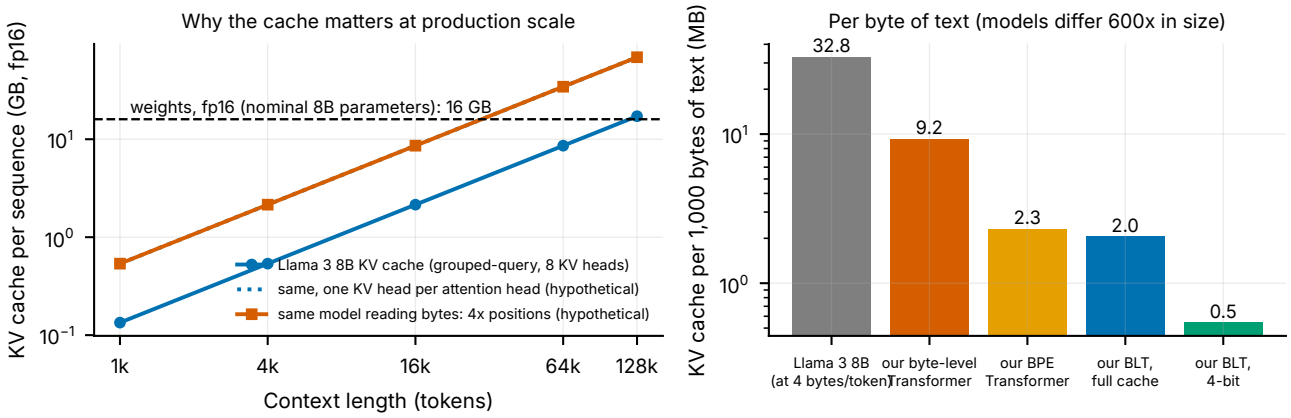


Figure 2. Left: KV cache of a production-scale model grows linearly with context and passes the size of its weights near 128k tokens (computed from the Llama 3 8B configuration: 32 layers, 8 KV heads of dimension 128; fp16). A byte-reading version of the same model would need four times as many positions (hypothetical). Right: our small models per 1,000 bytes of text; the production figure assumes 4 bytes per token and is not comparable in quality, only in cost structure. Configuration and sources in `configs/public_models.json`.

Table 2. Corpora. Bytes are after normalisation and deduplication. Evaluation sets are fixed and shared by every model. Counts for TinyStories and WikiText-103 are from `data/processed/*/stats.json` in the Part 2 release.

Corpus	Domain	Size and use
TinyStories	Synthetic children's stories	30,000 train stories (26.7 MB); 21,990 validation; evaluation on the first 500 validation stories (405,231 bytes); no test split
WikiText-103	English Wikipedia articles	29,002 train articles (526.4 MB, 442 duplicates removed); validation 60 articles (1,129,194 bytes); test 62 articles (1,269,744 bytes)
The Stack Smol	Python, Java, JavaScript files	28,229 files, 15% of each training batch in the code-mix models; 200 validation files
PG-19	Pre-1919 books	test split, 100 books (40.5 MB); first 100,000 bytes of 20 books, streaming only

cles (about 18 KB each), which suits long-context cache experiments. Both have a standard, citable form and the two sit at opposite ends of difficulty (our baselines reach 0.69 and 1.52 BPB). The Stack [12] (the Smol sample) adds code, because a byte model should not fail on code. PG-19 [20] supplies very long documents. The first three are used for training and validation; the Wikipedia test split was evaluated once at the end.

Limitations and bias. TinyStories is synthetic, narrow and repetitive; long training overfits it (37 epochs of a 26.7 MB file, Section 5.4). WikiText-103 is English and encyclopedic, so it reflects the topics and registers Wikipedia covers and under-represents conversational and non-English text. The Stack sample is permissively licensed code with only exact duplicates removed, so near-duplicate files may make validation code slightly easy. PG-19 is old literary English, far from the WikiText-trained models, so its BPB is high. The release copy of WikiText train is cut to 3,000 articles to keep the demo download at 138 MB (our choice, not a GitHub limit); all reported runs use the full corpus.

Preprocessing. WikiText articles were rebuilt from the raw line export, which has no article boundaries, and placeholder tokens such as `@-@` were restored. All text is NFC normalised and exact duplicates are removed. Each split is one uint8 file of UTF-8 bytes plus an offset array; documents are isolated by a BOS byte and an attention mask; code is split by content hash so duplicates never straddle splits. Every threshold and hyperparameter was tuned on validation only.

4 Method

Model. The model is a BLT-style network of 13,333,632 parameters. A local encoder (1 Transformer layer, width 256, 4 heads, rotary positions, causal window of 256 bytes) reads bytes. The encoder states of the bytes in a patch are mean-pooled and projected to width 384. A global Transformer (6 layers, width 384, 6 heads) runs causally over patches. A local decoder (2 layers, width 256, window 256) predicts each byte from its own encoder state and the global output of the patches that precede it. The global K and V tensors are the cache we compress: one entry per patch, $2 \cdot 6 \cdot 384 \cdot 2 = 9{,}216$ bytes at fp16. Step-by-step decoding equals the full forward pass to 10^{-5} (tested).

Patching. Entropy patching follows the BLT: a separate 3.3M-parameter byte model (4 layers, width 256, window 512) gives the next-byte entropy H_i, and a patch starts where H_i exceeds a threshold tuned on validation to give 4.5 bytes per patch (1.09 nats on TinyStories, 2.17 on WikiText). The decision uses only earlier bytes (tested). Fixed-stride patching (4 or 5 bytes) is the control. On the entropy patches 67.8% (TinyStories) and 66.7% (WikiText) of patch starts fall on word starts against 19.9% and 16.3% by chance.

Cache policies. A policy keeps at most B patch entries; the first patch (sink) and the 8 newest are always kept, and every policy chooses among the rest, so differences come from the signal and not from the window. Scores per old patch are:

- *recency*: newest first (StreamingLLM-style);
- *random*: three draws averaged;
- *keep high entropy* (the proposal): the mean entropy of the patch, $s_j = |P_j|^{-1} \sum_{i \in P_j} H_i$, keep the largest;
- *keep low entropy*: the opposite control;
- *H2O*: attention received so far, summed over queries, heads and layers;
- *SnapKV-style*: attention received from the last 16 patches only.

Entropy and recency policies are simulated by masks in

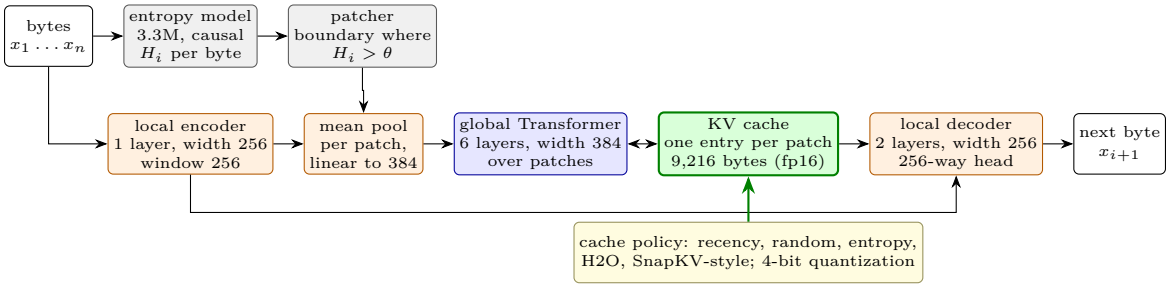


Figure 3. The BLT-style model and where the cache policy acts. Bytes are patched by the entropy model, encoded locally, pooled per patch and processed by a global Transformer whose keys and values are the cache; the local decoder predicts each byte from its own encoder state and the global output of earlier patches. The cache policy decides which patch entries are kept and at which precision.

one pass and tested equal to step-by-step decoding with entries really removed. H2O and SnapKV-style need the evicted cache, so `forward_online` runs the global layers patch by patch, chooses the entry to drop before the new patch is attended to, and accumulates attention as it is produced (tested for causality and for equality with the full cache at a large budget).

Quantization. Each K and V vector is stored with symmetric absmax scaling at b bits plus one fp16 scale per head. Bytes per entry are 9,216 (fp16), 2,448 (4-bit, 27%) and 1,296 (2-bit, 14%). Tier policies give a fraction of patches a high width (4 bits) and the rest a low one (2 bits), chosen by recency, random, or entropy, against uniform widths. Eviction and quantization combine by quantizing every kept entry.

Implementation and environment. Models, policies, quantizer and training loop are our own PyTorch code (`models/`, `kvc/`), with rotary positions [23], LayerNorm [1], GELU [8], a 4× MLP and pre-norm blocks [25]; training uses AdamW [16] and PyTorch [19]. No pretrained checkpoint is used; every weight is trained from scratch. Hyperparameters (BLT): AdamW, learning rate 10^{-3}, weight decay 0.01, 200 warm-up steps then cosine decay to 10%, gradient clip 1.0, bf16 autocast, batch 8 of 4,096 bytes, 3,000 steps (30,000 for the long runs), seed 0 to 2. Entropy model: learning rate $2 \cdot 10^{-3}$, batch 32 of 1,024 bytes, 3,000 steps, clip 1.0. Seed 0 of the prose-only models was trained on a laptop NVIDIA RTX 4060 (8 GB, Python 3.13, PyTorch 2.9) and the other runs on a cluster NVIDIA RTX PRO 6000 (95 GB); demos and the analyses in Section 5.6 run on CPU.

Streaming. For long documents the fixed-size cache runs 16,384-byte chunks (and, for PG-19, 100,000 bytes one byte at a time with `BLT.step`) against a baseline that resets the context every 4,096 bytes, the training length.

5 Experiments

5.1 Setup, metrics and baselines

Training budgets. The main models see about 98M bytes (3,000 steps); some WikiText code-mix models train for 30,000 steps (about 983M bytes, 1.9 epochs) and at 16,384-byte context. The same checkpoint scores identically on the laptop and the cluster GPU.

Metric. Bits per byte is the total negative log2-probability of the document divided by its byte count. It does not depend on the tokenizer, so the 4-gram, the BPE Transformer and the byte models are compared on identical text, and it equals the size per byte that an arithmetic coder driven by the model would reach [3]. Its limits: it measures prediction, not downstream use such as retrieval; differences between cache policies are small (at most 0.03 BPB), so every comparison is paired by document and reported with a 95% bootstrap interval over documents (2,000 resamples); and the Part 2 BPE number is 0.4 to 0.5% optimistic because the first token of each 256-token chunk is context only while its bytes are counted (corrected values 0.6978 and 1.5257). Cache size is the mean resident global KV size computed from entry counts and bit widths, and measured on a GPU only for recency (Figure 8).

Table 3. BPB on identical documents, lower is better. TS is TinyStories, WT is WikiText-103. First rows are the Part 2 baselines. BLT rows are seed 0, 3,000 steps, validation; the last row is the mean of the three 30,000-step code-mix seeds (1.2949, 1.2983, 1.3118), which saw ten times more training than the baselines.

Model	TS val	WT val	WT test
KN byte 4-gram	1.8085	2.4103	2.4381
BPE Transformer 13.8M	0.6939	1.5200	1.5309
Entropy model 3.3M	0.8438	1.6983	n/a
BLT, stride 4	0.7726	1.7600	1.7894
BLT, entropy patching	0.7980	1.6801	1.6986
BLT, entropy, 30k steps	n/a	1.2983	n/a

Baselines. Classical: interpolated Kneser-Ney byte 4-gram. Modern: a 13.8M BPE Transformer (6 layers, 6 heads, width 384, 8k vocabulary, 256-token context, 3,000 steps). Cache baselines: full cache, recency with a sink, random, H2O, SnapKV-style, and uniform 8, 4, 3 and 2 bits. The Transformer was trained before gradient clipping was added (clipping gives 0.6826 on TinyStories), so the modern baseline is slightly weak.

5.2 Baselines and base models

Table 3 gives the comparison. Every neural model is far ahead of the 4-gram (a gap of 1.0 to 1.1 BPB on TinyStories and 0.65 to 0.9 on WikiText). At equal training the BPE Transformer is better than our BLT-style model on both sets (0.69 against 0.80 on TinyStories, 1.52 against 1.68 on WikiText). This is a smaller, shorter-trained model without the hashed n-gram embeddings or cross-attention of the original BLT, so we do not claim a win over BPE at equal training. With ten times more training (a comparison that favours the BLT, since the Transformer kept 3,000 steps) the WikiText code-mix BLT reaches 1.29 to 1.31, and its cache is the one we can compress (below).

Entropy patching against fixed stride, three seeds each (mean full-cache BPB): on WikiText 1.6958 (entropy) against 1.7347 (stride 4) and 1.7629 (stride 5), with entropy patching lower in all three same-seed pairs and with fewer cache entries per byte than stride 4; on TinyStories 0.8183

Table 4. BPB of the seed-0 entropy-patched model under cache budgets (first patch and 8 newest protected). Full cache: 0.7980 (TinyStories), 1.6801 (WikiText). Share of full cache bytes in brackets.

Budget	recency	random	low ent.	high ent.
TinyStories 32 (32%)	0.8078	0.8115	0.8089	0.8231
TinyStories 64 (56%)	0.8016	0.8033	0.8026	0.8109
WikiText 64 (15%)	1.6836	1.6860	1.6901	1.6922
WikiText 128 (28%)	1.6814	1.6833	1.6864	1.6885

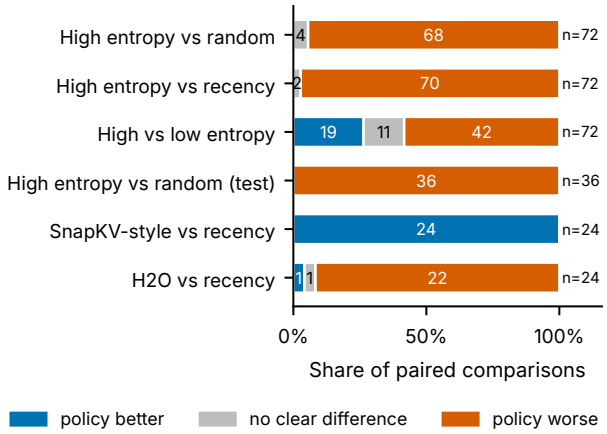


Figure 4. Scoreboard of paired comparisons, counted from the result tables. Keeping high-entropy patches is never better than random or recency; the SnapKV-style rule beats recency in every comparison; H2O mostly loses to recency.

against 0.8237 and 0.8521. Seed ranges are 0.056 and 0.028 for entropy patching and 0.135 and 0.044 for stride 4, so the WikiText gap of 0.039 is about the size of a seed range. We call it likely, not established. On TinyStories entropy patching ties stride 4. A first single-seed comparison that put entropy patching 3.3% behind stride 4 on TinyStories did not survive the seeds and was retired.

5.3 Entropy-guided eviction: a negative result

Table 4 and Figure 4 show the proposal losing to random and to recency at every budget. The pattern holds across the evidence we collected:

- Over 72 comparisons (9 models in each of 4 settings, 2 budgets) keeping high entropy is worse than recency with the interval above zero in 70 and indistinguishable in 2, worse than random in 68 and indistinguishable in 4, and never better than either. The paired difference against random is +0.0003 to +0.0245 BPB.
- It holds in every seed, for fixed-stride models (so it is not caused by uneven entropy patches), with the code mix, on the WikiText test split (all 36 model and budget cells worse than random and recency), at 16,384 bytes (+0.016 to +0.021 against random at 64 to 512 patches), and for a recency-protected variant (newest half protected, other half by entropy: worse than recency in 23 of 24 cells).
- Ablations do not rescue it (three seeds): the model's own entropy, the first-byte score, the mean score and sink sizes 0 and 4 are worse than random in all seeds on both datasets; the maximum rule is worse in 3 of 3 (WikiText) and 2 of 3 (TinyStories) seeds; only the sum rule on TinyStories is ever better (one seed, −0.0024).
- Keeping *low* entropy beats keeping high entropy in 42 of 72 comparisons, is worse in 19 and ties in 11, so low-entropy retention is not a dependable alternative either.
- A byte-level cache (one entry per byte) loses much more than a patch cache at equal entry counts: at 32 entries recency costs +0.2286 BPB against +0.0133 with patches. Patches win per byte of cache; bytes win only with about 4.5 times more cache.

Table 5. Eviction with uniform 4-bit K and V, mean of three seeds, cost over the full fp16 cache in BPB (share of full cache bytes). Full cache: 0.8183 (TinyStories), 1.6958 (WikiText).

Setting	TinyStories	WikiText
SnapKV-style, 64, 4-bit	+0.0049 (15%)	+0.0026 (3.9%)
recency, 64, 4-bit	+0.0072 (15%)	+0.0046 (3.9%)
keep high ent., 64, 4-bit	+0.0224 (15%)	+0.0196 (3.9%)
SnapKV-style, 128, 4-bit	n/a	+0.0012 (7.4%)
recency, 128, 4-bit	n/a	+0.0019 (7.4%)

5.4 More training makes the penalty larger

A natural objection is that a small, short-trained model reads little from its cache. WikiText code-mix models trained for 30,000 steps score 1.2949, 1.2983 and 1.3118 BPB and keeping high entropy costs +0.0287, +0.0284 and +0.0238 against random at 64 patches (against +0.0020 at 3,000 steps), with every interval excluding zero (Figure 5). Any eviction also costs more (recency at 64 patches costs 0.018 BPB over the full cache, from 0.002), and the fixed-stride model shows the same sign. The TinyStories version overfits (training loss 0.14 nats, validation BPB 1.6493) and is not used. A longer context does not rescue it either: our first 16,384-byte models were under-trained and suggested the penalty fades, a conclusion withdrawn after tuning (Section 6).

5.5 What works: attention, quantization, bounded memory

Attention-based scoring. Online, the SnapKV-style rule is better than recency in all 24 comparisons on the 12 entropy-patched models (by 0.0002 to 0.0077 BPB, for example −0.0013 [−0.0018, −0.0009] on WikiText code mix, seed 0, 64 patches), and never worse than its offline approximation. With offline scoring it is also better than recency in 33 of 36 test-split cells (the other three tie) and, for the 30,000-step models, by 0.0051, 0.0057 and 0.0049 at 64 patches. Online H2O is worse than recency in 22 of 24: accumulated attention favours old patches that were popular early. The effect is small (for seed 0 on TinyStories at 32 patches recency costs 0.0098 over the full cache and SnapKV-style recovers 0.0041), but it is the one signal that beat recency, and it is attention, not surprise.

Quantization and combination. Uniform 4-bit K and V costs 0.0003 to 0.0014 BPB over fp16 across ten models at 27% of the bytes (uniform 3-bit 0.0014 to 0.0077, 2-bit 0.022 to 0.111). Entropy-guided precision tiers lose to random and to recency tiers in all ten models. Eviction and 4-bit together, mean of three seeds, are in Table 5: a WikiText cache of 128 patches in 4-bit holds 7.4% of the full cache bytes and costs +0.0012 BPB with SnapKV-style eviction (+0.0019 with recency), while keeping high entropy costs +0.0151. No setting that keeps high-entropy patches is on the bytes-against-BPB frontier (Figure 6).

Bounded-memory streaming. Three WikiText code-mix models trained on 4,096-byte chunks decoded 46 validation documents longer than 8,192 bytes with a fixed

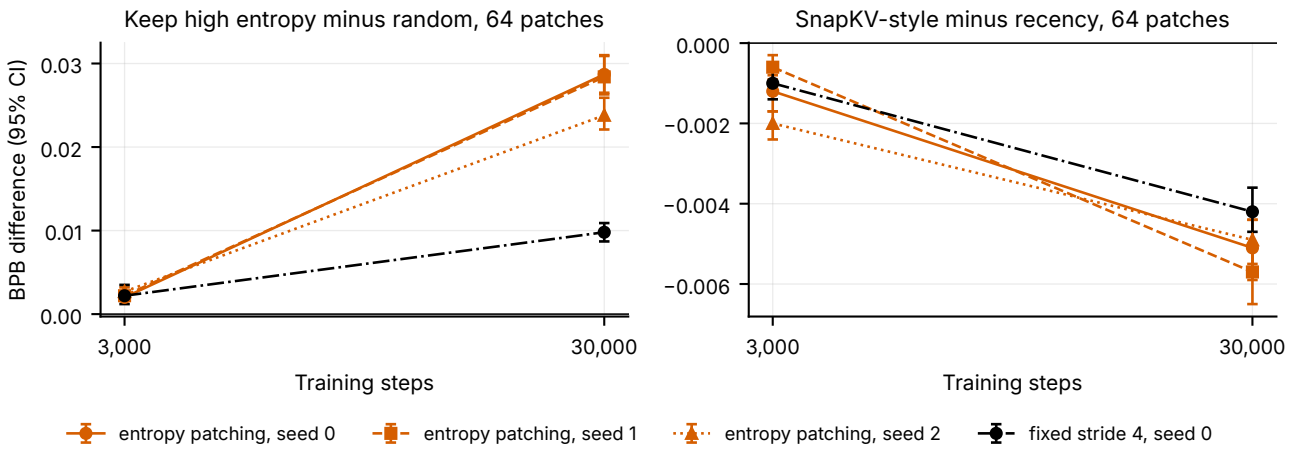


Figure 5. Ten times more training (3,000 to 30,000 steps, WikiText with code mix) makes the entropy penalty about ten times larger (left) and the SnapKV-style gain several times larger (right). Bars are 95% bootstrap intervals; three seeds of entropy patching, one of fixed stride.

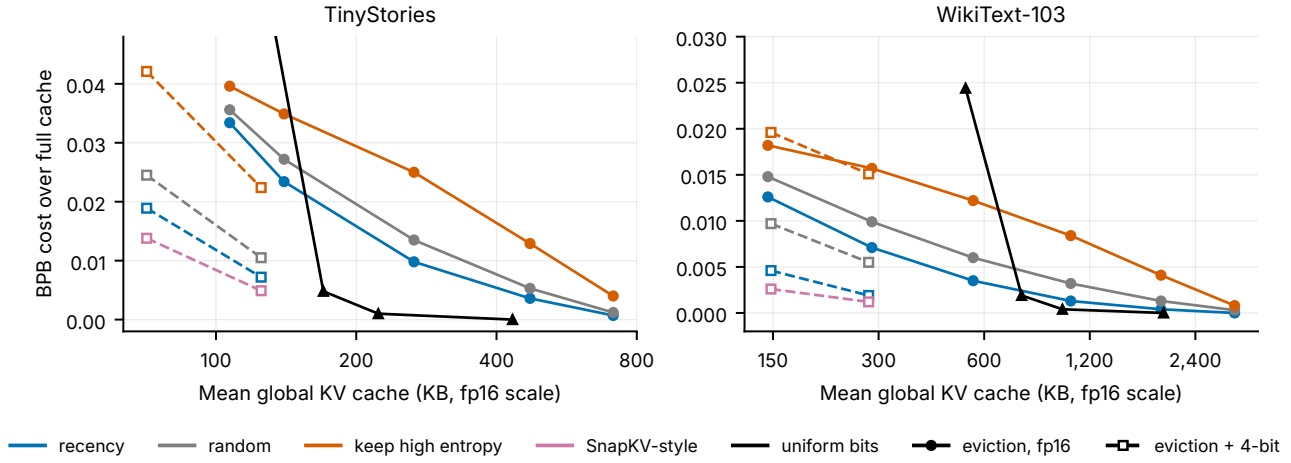


Figure 6. Cost over the full cache against cache size (seed-0 eviction and uniform rows from `comparison.md`; 4-bit rows are the three-seed mean from `deploy_combo.md`). Keeping high entropy (orange) is the worst eviction rule at every size and never on the frontier; SnapKV-style with 4-bit (pink, hollow) is the best low-memory point. Uniform 2 bits is off scale.

patch budget. Recency with 64 patches beats the chunked full-cache baseline in all three seeds (by 0.0030, 0.0032 and 0.0008 BPB; with 256 patches by 0.0051, 0.0058 and 0.0035), because the reset loses the context at every chunk boundary and a small continuous window does not. SnapKV-style is better still at 64 patches (0.0014, 0.0008 and 0.0024 below recency). Random, H2O and keeping high entropy are worse than recency at every budget in all seeds (Figure 7, left). On PG-19, with the 30,000-step model, 256 patches of recency beat the reset baseline by 0.0086 BPB [0.0079, 0.0093] and the whole-book full cache by 0.145; the whole-book full cache is worse than the reset baseline (by 0.137), so the model does not use a context 24 times longer than it was trained on. Keeping high-entropy patches is +0.093 to +0.122 worse than recency. Keeping old patches means attending at distances never seen in training, and we did not renumber positions inside the cache as StreamingLLM does, so PG-19 cannot separate a poor entropy signal from poor long-distance attention.

Memory and speed. Figure 8 gives the measured savings. Fixed costs are not reduced: the 786 KB byte-window caches and the entropy model (about 2.1 MB of window cache, 3.3M parameters), so at 64 patches they exceed the global cache. At full precision the patch cache is about 0.9 times a BPE Transformer cache of the same width, and a byte-level Transformer needs about four times more.

Compute per byte. The analytic count in the supplement (Table S1, `results/part3/flops.md`) is not a measurement. The BLT is cheaper than a byte-level Transformer (11.1 against 30.9 MFLOPs per byte at a context of 1,024 bytes), but it is *more* expensive than the BPE Transformer (7.4), because its local encoder and decoder run on every byte, and it needs the separate entropy model (8.5 MFLOPs per byte) when patches come from entropy; with it the total is 19.6. A 64-patch cache trims only the global attention term (0.3 MFLOPs at 1,024 bytes, 1.7 at 4,096). So the case for the BLT here is cache memory, not FLOPs.

Retrieval. In a planted-name test (a random name repeated after 400 to 3,000 filler bytes) the models score the same with and without the earlier mention, even at 30,000 steps, so no policy can be shown to help and nothing about retrieval can be concluded. The entropy rule does keep the name's patches more often than random (45% against 31% at 64 patches and 400 bytes), but the model gains nothing from them.

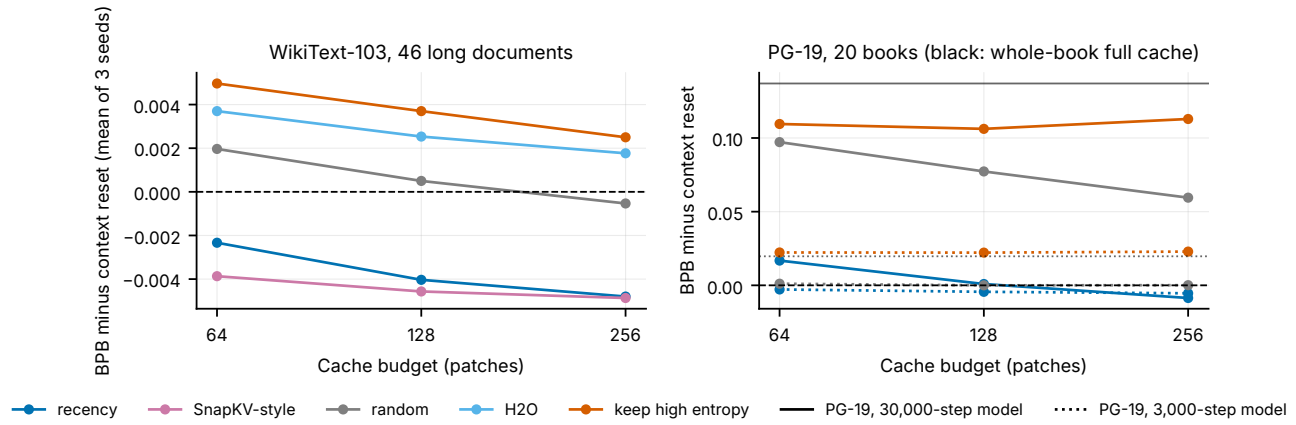


Figure 7. Bounded-memory streaming. Left: WikiText documents longer than 8,192 bytes decoded in 16,384-byte chunks (patch positions four times beyond training); below zero beats resetting the context every 4,096 bytes. Right: 20 PG-19 books of 100,000 bytes decoded one byte at a time (positions about 22,000). Recency is the only policy that is safe on books; keeping high entropy is worst but is confounded by distance (Section 5.5).

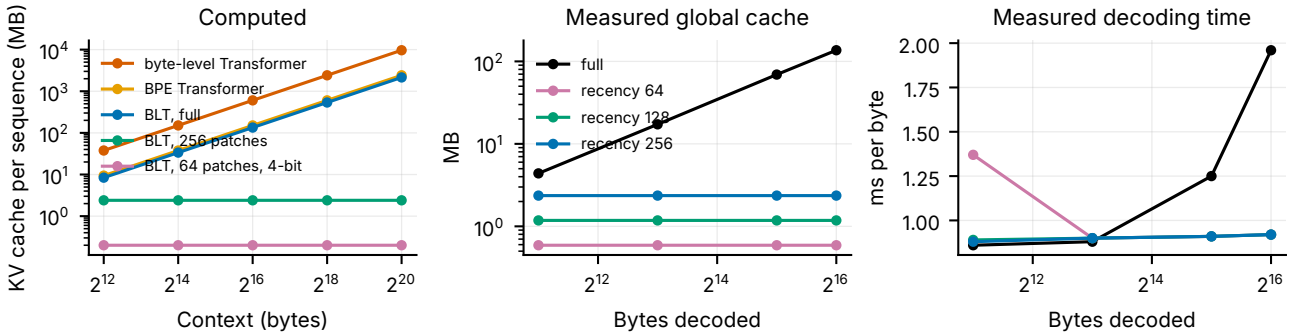


Figure 8. Cache memory. Left: computed KV size per sequence. Middle and right: measured on a GPU, one stream, real eviction of recency. A bounded cache holds the global cache at 0.59 to 2.36 MB while a full cache reaches 136.5 MB at 65,536 bytes; time per byte stays at 0.9 ms against a rise to 2.0 ms. The first recency point includes warm-up.

5.6 Error analysis

Part 1 plan. The proposal promised a split of BPB by entropy, a code against prose comparison, a retrieval probe, and a look at generations where the compressed and full models diverge. All four are done: Table 6 and Figure 9, `code_domain.md` (prose-only models score 7.2 to 7.6 BPB on code, the 15% code mix 1.75, and entropy retention is still worse than random there, +0.0036), the planted-name test (uninformative), and Example 5 below. We scored 100 TinyStories documents (74,760 bytes, budget 32 patches) and 30 WikiText documents (122,880 bytes, 64 patches) with the seed-0 models on a CPU (`docs/error_analysis_part3.md`, generated by `scripts/error_analysis_part3.py`). The documents are the first ones that pass a length filter; the tables describe one seed and are not tests.

Quantitative. Table 6 splits the extra loss by the entropy tercile of the patch holding the byte. Every rule loses most on the high-entropy third, where the full-cache loss is also highest, but keeping high-entropy patches does not protect those bytes (+0.0392 against +0.0225 for recency on TinyStories; +0.0190 against +0.0062 on WikiText), and it also costs the easy bytes (+0.0194 against +0.0080 and +0.0152 against +0.0023 on the lowest third, which holds 57% and 55% of the bytes). The entropy rule keeps almost only high-entropy patches (44.6% and 18.1% of the highest third survive, none of the lowest) and they are short: the mean kept patch is 1.15 and 1.13 bytes long and

Table 6. Extra loss over the full cache (BPB) by entropy tercile of the patch holding the byte. Share of bytes in brackets.

Set	Tercile	recency	high ent.	low ent.
TinyStories	lowest (57%)	+0.0080	+0.0194	+0.0065
	middle (32%)	+0.0104	+0.0272	+0.0130
	highest (11%)	+0.0225	+0.0392	+0.0253
WikiText	lowest (55%)	+0.0023	+0.0152	+0.0035
	middle (31%)	+0.0037	+0.0146	+0.0165
	highest (14%)	+0.0062	+0.0190	+0.0274

98.0% and 99.7% of them have one or two bytes, against 4.46 and 4.47 bytes for recency. This is a length confound (a surprising byte often forms its own patch, so a mean over a short patch is high). It does not explain the whole result, since the sum rule, which favours long patches, ties random on TinyStories and loses on WikiText (Section 5.3). By byte class, the entropy rule loses most on word-initial lowercase letters (+0.0437 against recency on WikiText) and on capitals in TinyStories (+0.1426), and gains a little on digits and capitals in WikiText (−0.0199 and −0.0083). The damage is concentrated: the largest 1% of bytes carry 30.4% (TinyStories) and 18.0% (WikiText) of the extra loss against recency, and entropy is worse than recency on 40.7% and 51.0% of bytes and better on 33.1% and 42.4%.

Why? Attention. Figure S1 of the supplement (`report/supplement.pdf`) shows the Spearman correlation between the entropy of an old patch (at least 8 patches back) and the attention it receives. It is negative in the middle and upper layers of most models: in all six lay-

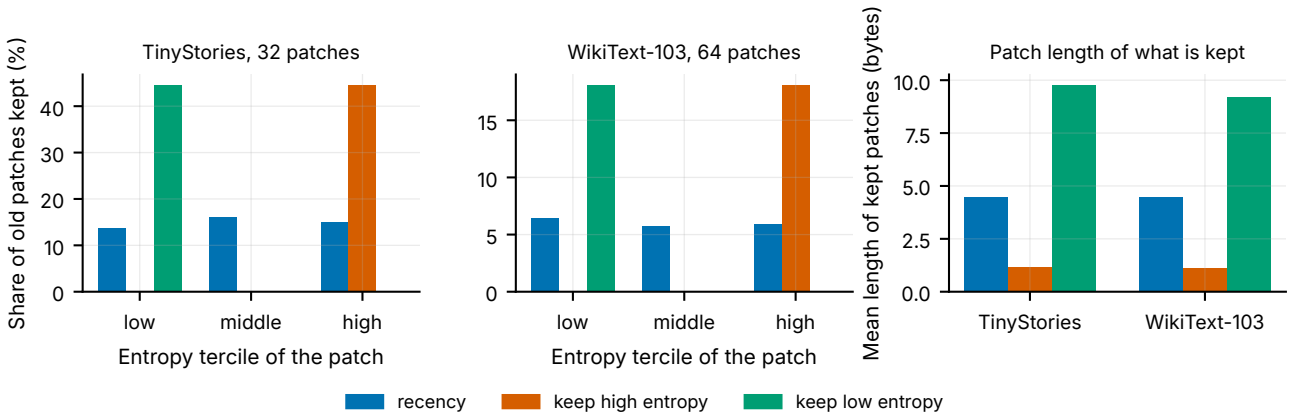


Figure 9. What each rule keeps at the end of a document. Left and middle: share of old patches kept, by entropy tercile; keep-high-entropy keeps only high-entropy patches (and keep-low-entropy the opposite). Right: mean length of the kept patches; the entropy rule keeps one-byte fragments.

ers of all three TinyStories seeds (−0.04 to −0.55), and in layers 2 to 5 of the three prose-only WikiText seeds (−0.007 to −0.39). In the seed-0 TinyStories model the highest-entropy third of old patches receives about half the attention of the lowest third in layers 1 and 2. The model reads predictable patches more than surprising ones, and surprising bytes are handled by the local windows. This is a correlation, not an intervention, and the WikiText code-mix seed 0 model shows the opposite sign in most layers while its eviction result has the same sign, so it is not a complete account.

Qualitative. In the examples | marks a patch boundary; all are real output.

Example 1, TinyStories document 0 (1,201 bytes, 4.0 bytes per patch). Boundaries fall before surprising letters and split names: `R|o|xy. |Ro|x|y |loved to |c|limb.`, while predictable words form whole patches (`She |had |never |seen |anything |`). At the last patch the caches (budget 32) hold, for recency, `played |together |all the |time. |And |Ro|x|y |l|earned that |...` (BPB 0.744, +0.010 over full); for keep high entropy, `|s|f| |B|s|s|Ro| |b|f|b|f|a|s|a|i|...` (0.751, +0.017): a string of one-letter fragments that keeps `Ro` but not `Roxy`; for keep low entropy, `Once upon a time|, in a |forest, there li|...|never |seen |ried |again, |...` (0.743, +0.009), whole words from the start of the story.

Example 2, WikiText document 0 (4,096 bytes, budget 64). The entropy cache at the last patch is `o|Ho|E|A|O|S|A|a|6| |a |e|5 |1|e|e|1|...`: capitals, digits and vowels, no words. BPB is 1.752 (+0.033) against 1.725 for recency (+0.006), whose cache holds the section on distribution (`Norway |to |the |A|zores |and |Mo|r|occo`).

Example 3, a locally predictable byte. In TinyStories document 56 the byte `d` in `They argue[d]` costs 0.02 bits with the full cache, 0.01 with recency and 5.97 with keep high entropy. The recent bytes are present (the newest 8 patches are protected), so what changes is the older cache; we hypothesise that a cache of fragments distorts the global state that feeds the decoder even for easy bytes, but we did not test it. A WikiText case: `to the northeast , i[s]` costs 0.84, 0.86 and 4.15 bits.

Example 4, where entropy helps. In TinyStories document 13 the byte `S` in `All the animals cheered for [S]am` costs 4.04 bits with the full cache, 7.35 with recency and 3.46 with keep high entropy. The name Sam appears several times earlier in the story. We did not check whether the entropy cache held a patch with the name. Such cases exist but are outweighed on average.

Example 5, generations that diverge. We continued the same prompt greedily with the full cache and with 32 (TinyStories) or 64 (WikiText) patches (`results/part3/generation_divergence.md`). On TinyStories document 0 the full cache continues `find your friends." Roxy was so happy to help Roxy.`; recency diverges at byte 17 (`find your friends to climb the leaves.`) and random and keep low entropy at byte 10 (`find your home and they would...`); keep high entropy stays on the full-cache text until byte 25 (`Roxyy said, "Thank you, Billy.`). On WikiText document 1 recency stays with the full cache until byte 65, whereas random and keep high entropy diverge at the first byte (`r the 1970 - 1996 season`). Greedy text from such small models is repetitive (`the state of the state`), so we read these as evidence of where the compressed cache changes the prediction, not as quality measures: on these four prompts no policy reproduces the true continuation better than the full cache by more than a few bytes, and the examples are anecdotal.

6 Challenges and how we handled them

No separate Part 1 problems were recorded; the table covers Parts 2 and 3 (full entries in `CHALLENGES.md`). Most were found by our own consistency checks.

7 Scope and limitations

- **Scale.** Models have 13.3M parameters and mostly 3,000 training steps. The 30,000-step models show the same sign with larger effects, but a much larger model could behave differently. Our models do not retrieve (planted-name test), so tasks that need retrieval are untested.
- **Data and measurement.** TinyStories has no test split and no usable long-training result; PG-19 uses 20 books and out-of-domain models. Cache sizes are computed from entry counts, only recency eviction was

Table 7. Challenges, what we did, and the outcome.

Challenge	What we did	Outcome
Part 2 BPB counted the first token of each chunk, which is context only	Kept the reported numbers for traceability, disclosed the correction (0.6978 and 1.5257), score every byte in Part 3	BPE number is 0.4 to 0.5% optimistic; documented, not hidden
Seed spread as large as some gaps (0.056 to 0.135 BPB)	Three seeds for every patching scheme, a stride-5 control, paired bootstrap by document	A single-seed 3.3% claim retired; the eviction sign holds in every seed
Cache byte shares were halved by a counting error	Found when eviction and quantization rows were put in one table (1,669 KB against 839 KB); fixed, all evaluations rerun, tables regenerated	BPB unaffected; shares corrected
First 16,384-byte models under-trained (1.89 BPB)	Tuned learning rate and batch size on validation (1.5843)	Penalty does not fade with context; earlier reading retired. Context length and training amount are not separated
TinyStories overfits at 30,000 steps (37 epochs)	Used the WikiText scale-up (1.9 epochs) as the evidence	TinyStories rows reported for completeness only
Code data gated; prose models score 7 to 8 BPB on code	Team member login built the set; retrained with a 15% code mix	Eviction conclusion unchanged; mix costs about 0.05 (WikiText) and 0.09 (TinyStories) BPB
CUDA out-of-memory left a half-written entropy cache while the queue carried on	Caught from the log, cache rebuilt, queue now stops if the cache script fails	No training on corrupt data
Offline attention scores only approximate online policies	Built and tested `forward_online`	Online SnapKV-style better than recency in 24 of 24
Models do not retrieve (planted-name test)	Reported retention rates, drew no retrieval conclusion	Scope stated honestly
PG-19 ships as a loading script current libraries do not run	Used a community mirror of the same test split, recorded the substitution	20 books streamed byte by byte
One job at a time, 2-day limit	Checkpoint, handle the pre-limit signal, requeue, resume	Verified end to end under one job id (signal at step 1,864, finish at 24,000)

measured on a GPU, quantization is simulated by rounding, FLOPs are analytic.

- **Seeds.** The 30,000-step fixed-stride model and the 16,384-byte models have one seed; streaming uses three seeds of one setting; seed 0 of the prose-only models trained on a different GPU (the same checkpoint scores identically on both).
- **Mechanism.** The attention and fragment accounts of why entropy fails are correlational; no intervention separates a length effect, an entropy effect and a distortion of the global state. On PG-19, keeping old patches also means attending far beyond the training range, and H2O and SnapKV-style were not run there.

8 Conclusion

New. A controlled test of entropy-ranked cache retention on a byte-level patch model, with seeds, paired intervals and a test split; an online, tested implementation of attention policies over patch caches; and measured evidence that a fixed recency or SnapKV-style cache streams documents several times longer than the training context better than resetting it. **Improved.** Eviction plus 4-bit quantization keeps 7.4% of the cache for +0.0012 to +0.0019 BPB; SnapKV-style eviction beats recency in 24 of 24 comparisons; memory and time per byte stay constant with a bounded cache; the BLT needs 2.8 times fewer FLOPs per byte than a byte-level Transformer. **Did not work.** The entropy that places patch boundaries is not a good signal for which patches to keep, and more training makes it worse; the error analysis shows it keeps mostly one- and two-byte fragments. **Open.** Whether the results hold for much larger models, or for tasks that need retrieval.

Demo and reproducibility. A CPU demo (`scripts/demo.py`) compresses a text, trains a small byte-level model live and streams a long text with a real fixed-size cache (`docs/demo_script.md`). Every table and figure regenerates from the repository README commands and `report/make_figures.py`.

References

[1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

[2] Stanley F. Chen and Joshua Goodman. An empirical study of smoothing techniques for language modeling. *Computer Speech and Language*, 13 (4), 1999.

[3] Grégoire Delétang, Anian Ruoss, Paul-Ambroise Duquenne, Elliot Catt, Tim Genewein, Christopher Mattern, Jordi Grau-Moya, Li Kevin Wenliang, Matthew Aitchison, Laurent Orseau, Marcus Hutter, and Joel Veness. Language modeling is compression. *arXiv preprint arXiv:2309.10668*, 2023.

[4] Ronen Eldan and Yuanzhi Li. TinyStories: How small can language models be and still speak coherent English? *arXiv preprint arXiv:2305.07759*, 2023.

[5] Stephen Gould and Anton van den Hengel. Back from the future: Key-value cache management by counter-causal surprise. *arXiv preprint arXiv:2607.27600*, 2026.

[6] Aaron Grattafiori, Abhimanyu Dubey, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[7] Sayed Pedram Haeri Boroujeni, Niloufar Mehrabi, Patrick Woods, Gabriel Hillesheim, and Abolfazl Razi. Don't waste bits! adaptive KV-cache quantization for lightweight on-device LLMs. *arXiv preprint arXiv:2604.04722*, 2026.

[8] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (GELUs). *arXiv preprint arXiv:1606.08415*, 2016.

[9] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W. Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. KVQuant: Towards 10 million context length LLM inference with KV cache quantization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2401.18079.

[10] Jushi Kai, Zhuiri Xiao, Alexandra Birch, and Zhouhan Lin. Information-aware KV cache compression for long reasoning. *arXiv preprint arXiv:2606.26875*, 2026.

[11] Reinhard Kneser and Hermann Ney. Improved backing-off for M-gram language modeling. In *IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, 1995.

[12] Denis Kocetkov, Raymond Li, Loubna Ben Allal, Jia Li, Chenghao Mou, Carlos Muñoz Ferrandis, Yacine Jernite, Margaret Mitchell, Sean Hughes, Thomas Wolf, Dzmitry Bahdanau, Leandro von Werra, and Harm de Vries. The stack: 3 TB of permissively licensed source code. *arXiv preprint arXiv:2211.15533*, 2022.

[13] Yubo Li and Yidi Miao. CONF-KV: Confidence-aware KV cache eviction with mixed-precision storage for long-horizon LLM. *arXiv preprint arXiv:2605.24786*, 2026.

[14] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. *arXiv preprint arXiv:2404.14469*, 2024.

[15] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. In *International Conference on Machine Learning (ICML)*, 2024. arXiv:2402.02750.

[16] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019. arXiv:1711.05101.

[17] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. *arXiv preprint arXiv:1609.07843*, 2016.

[18] Artidoro Pagnoni, Ram Pasunuru, Pedro Rodriguez, John Nguyen, Benjamin Muller, Margaret Li, Chunting Zhou, Lili Yu, Jason Weston, Luke Zettlemoyer, Gargi Ghosh, Mike Lewis, Ari Holtzman, and Srinivasan Iyer. Byte latent transformer: Patches scale better than tokens. *arXiv preprint arXiv:2412.09871*, 2024.

[19] Adam Paszke, Sam Gross, Francisco Massa, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, 2019. arXiv:1912.01703.

[20] Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, and Timothy P. Lillicrap. Compressive transformers for long-range sequence modelling. *arXiv preprint arXiv:1911.05507*, 2019.

[21] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th*

Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Berlin, Germany, 2016. doi: 10.18653/v1/P16-1162.

[22] Kevin Slagle. SpaceByte: Towards deleting tokenization from large language modeling. *arXiv preprint arXiv:2404.14408*, 2024.

[23] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021.

[24] Hugo Touvron, Louis Martin, Kevin Stone, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

[25] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30*, 2017. arXiv:1706.03762.

[26] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2309.17453.

[27] Linting Xue, Aditya Barua, Noah Constant, Rami Al-Rfou, Sharan Narang, Mihir Kale, Adam Roberts, and Colin Raffel. ByT5: Towards a token-free future with pre-trained byte-to-byte models. *Transactions of the Association for Computational Linguistics*, 2022. arXiv:2105.13626.

[28] Lili Yu, Dániel Simig, Colin Flaherty, Armen Aghajanyan, Luke Zettlemoyer, and Mike Lewis. MEGABYTE: Predicting million-byte sequences with multiscale transformers. *arXiv preprint arXiv:2305.07185*, 2023.

[29] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023. arXiv:2306.14048.

[30] Lin Zheng, Vasilisa Bashlovkina, Timothy Dozat, Dan Garrette, Laura Rimell, and Joshua Maynez. Scratchpad patching: Decoupling compute from patch size in byte-level language models. *arXiv preprint arXiv:2605.09630*, 2026.